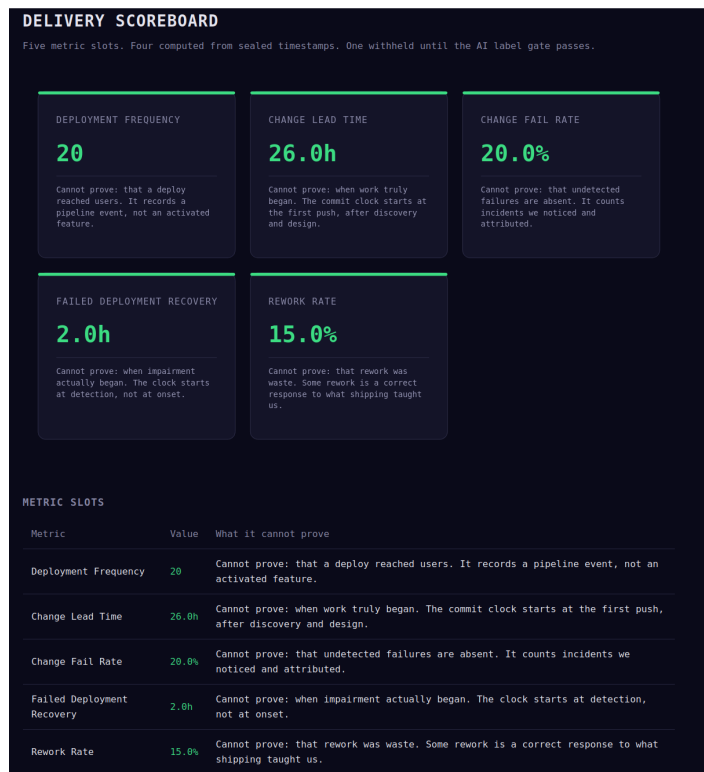


DORA DELIVERY SCOREBOARD WITH DUCKDB

By Ahmed Tetteh | September 2026



Presenting an AI-Governed Delivery Scoreboard

Reviewing the completed delivery readout

In this step, I'm reviewing the computed output from the scoreboard and the generated readout and verify every annotation is present and specific.

After-action insights from the AI gate

The gate returned WITHHELD initially (missing inspector), then PASSED (15.0%) once `ai_labels.json` arrived. The predicted count was 3 incident-triggered deploys out of 20. I found that the gate is a quality checkpoint that cannot run its accuracy verification without the AI labels file—it checks whether the model reproduces at least 18 of 20 sealed reference judgments, and without that data, it holds the metric rather than estimating it.

Measuring Rework with an AI Quality Gate

Classifying delivery rework

In this step, I'm classifying 20 deploy records for the AI client classification.

```
• (.venv) king@king-TFG5577XG:~/Desktop/Personal-Lab/Build & Brew Projects/delivery-scoreboard$ python scoreboard.py
diagnostic | median hours to production by start clock -- committed: 26.0h | PR-opened: 18.0h | merged: 1.0h
deployment_frequency: 20
change_lead_time: 26.0h
change_fail_rate: 20.0%
failed_deployment_recovery_time: 2.0h
rework_rate: 15.0%

wrote scoreboard.html
```

Gate outcome, incident prediction, and metric results

My gate returned PASSED, with rework_rate is reported as 15.0% (not withheld).

My AI predicted 3 incident-triggered deploys (d-006, d-011, d-015). The five metrics were:

deployment_frequency: 20, change_lead_time: 26.0h, change_fail_rate: 20.0%,
failed_deployment_recovery_time: 2.0h, rework_rate: 15.0%

Defining Delivery Performance Metrics in SQL

Building the metric calculations

In this step, I'm setting up the data in DuckDB and generate the SQL queries, so that I can audit it easily by reading a WHERE clause.

Why deployment clock definitions change the result

The clock definition matters because at different points in the same process, they measure different things, not just different numbers.

```
• (.venv) king@king-TFG5577XG:~/Desktop/Personal-Lab/Build & Brew Projects/delivery-scoreboard$ python scoreboard.py
diagnostic | median hours to production by start clock -- committed: 26.0h | PR-opened: 18.0h | merged: 1.0h
deployment_frequency: 20
change_lead_time: 26.0h
change_fail_rate: 20.0%
failed_deployment_recovery_time: 2.0h
rework_rate: WITHHELD (no AI labels found)

wrote scoreboard.html
```

Building a Verified Deployment Corpus

Creating the scoreboard's source data

In this step, I'm creating the corpus, so that the scoreboard can draw from different evidence.

Identifying incident-triggered rework

The incident-triggered deploys are d-006, d-011, and d-015. I can identify them by looking at the boolean values from the sealed labels.json and the commit message hotfixes in the deploys.json file

```
• (.venv) king@king-TF6577X6:~/Desktop/Personal-Lab/Build & Brew Projects/delivery-scoreboard$ python -c "import json; d=json.load(open('data/deploys.json')); i=json.load(open('data/incidents.json')); s=json.load(open('data/sealed_labels.json')); print(len(d), 'deploys,', len(i), 'incidents,', sum(1 for x in s if x.get('is_incident_triggered')), 'incident-triggered')"
```

Establishing a Documented Project Foundation

Setting up the project and decision documentation

In this step, I'm setting up the project folder, installing DuckDB and creating the pre-built skeleton for the collector.

Evaluating an architectural decision and reversal trigger

In ADR-004, I chose 30-day window / 20-deploy minimum over a 90-day window with a larger deploy minimum. The reversal trigger is if the team commits to quarterly review, switch to the 90-day window (with a correspondingly larger minimum) — the 30-day choice is explicitly tied to the scoreboard staying a point-in-time snapshot rather than a trend tool.

```
(.venv) king@king-TFG5577X6:~/Desktop/Personal-Lab/Build & Brew Projects/delivery-scoreboard$ git log --oneline
2dcdbd6 (HEAD -> master) Design: requirements, component view, build plan, four ADRs
06c57e8 Initial setup: skeleton, tasks, directories
```

Setting the Goal for the Delivery Scoreboard

Project purpose and intended outcome

In this project, I'm building a DORA delivery scoreboard that computes a 30-day history data, with AI-gated classification integrated within it, and a final output as a single HTML file.

Extending the Scoreboard for Production Evidence

Delivery evidence available from Git history

In this project extension, Git could prove commit timestamps and author identity, showing when work was first pushed. But it could not prove when pull requests were opened, when they merged to main, when code deployed to production, which incidents occurred, or when incidents were detected and resolved—the four timing fields needed for lead time, fail rate, and recovery time metrics.

Systems needed for a production-ready scoreboard

In this project extension, would connect a deployment tracking system (CI/CD platform) to obtain deployment timestamps, a code review API (GitHub/GitLab) to obtain PR open and merge times, and an incident management system (PagerDuty/Incident.io) to obtain incident detection and resolution timestamps. A post-incident review database would establish causal links between specific deploys and incidents—currently the hardest signal to automate with confidence.

Reflecting on Tools, Learning, and Next Steps

Key tools and concepts applied

The key tools I used include DuckDB (for SQL queries over JSON data), Python (for the metrics pipeline), Git (for commit inspection and version control), SQL (for metric definitions), and SVG/HTML/CSS (for the visual output).

Key concepts I learnt include how sealed labels are used as ground truth (using hand-audited reference data to validate AI predictions), AI gating (using a 90% accuracy threshold to decide whether to report a metric), and the four DORA metrics and their inherent blind spots.

Project effort and challenge

This project took me approximately 1 hr. The most challenging part was generating sample production deployment data that mimics real infrastructure and grabbing meaning out of the entire python pipeline.

Skills developed and future learning goals

I did this project today to learn how teams in production analyze their deployment data to streamline some bottlenecks they realize within the data.